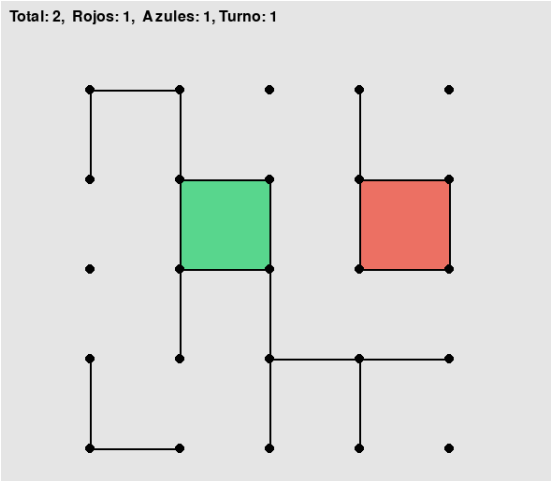
Universidad del País Vasco / Euskal Herriko Unibertsitatea

Facultad de Ciencia y Tecnología

Dots and Boxes - Implementación de Algoritmo Minimax



Rubén López de Juan, Unai Zuazo Angulo

Abril 2024

Índice

1. Introducción **2**
1.1. Cómo jugar 2
1.2. Estrategias Habituales 2

2. Ejecución **3**

3. Complejidad Temporal Teórica **5**
3.1. Función *pos_moves()* 5
3.2. Función *eval()* 5
3.3. Función *Minimax()* 5

4. Complejidad Espacial Teórica **6**

5. Análisis y Discusión **7**
5.1. Tiempo 7
5.2. Profundidad 10
5.3. Función *eval* 13

6. Conclusiones **13**

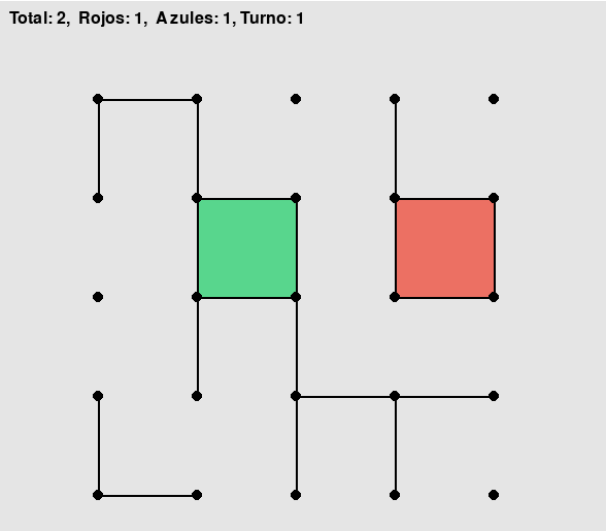
7. Anexo: Código Python **14**

1. Introducción

El objetivo de esta práctica es implementar el juego *Dots and Boxes* en Python, con el propósito de estudiar la optimalidad de un algoritmo *Minimax* para su desarrollo. Se compararán diferentes estrategias para ver cuál resulta mejor dependiendo del caso a trabajar.
Se trata de un juego muy conocido por no requerir más material que un papel y un lápiz, pues dos jugadores jugarán en una cuadrícula de puntos. Pese a sus pocas reglas y simple apariencia, su jugabilidad ofrece desafíos estratégicos que requieren pensamiento táctico y anticipación. Los jugadores compiten por conectar puntos adyacentes para formar líneas y, eventualmente, completar cuadrados. Cada cuadrado completado les otorga puntos, los cuales acumularán hasta el fin de la partida y ganará el que más consiga.

1.1. Cómo jugar

Se juega en una cuadrícula de puntos, generalmente de tamaño reducido, pues las partidas pueden durar mucho de no ser así. Cada jugador toma turnos para conectar puntos adyacentes (horizontales o verticales), uno a la vez, intentando completar un cuadrado. Cuando un jugador completa un cuadrado, suma un punto y conecta otros puntos diferentes, pudiendo así completar otro cuadrado y repetir el proceso. El objetivo es completar la mayor cantidad posible de cuadrados. Cuando se hayan conectado todos los puntos posibles, el juego termina y el jugador con más puntos gana.



1. Muestra de una partida de Dots and Boxes

1.2. Estrategias Habituales

Cada jugador desarrolla sus propias estrategias, pero las más habituales no suelen ser muy extravagantes. Una buena estrategia en general es no conectar puntos en un cuadrado que ya cuenta con dos aristas, pues en el siguiente turno el adversario podrá completar un cuadrado. En general, con este procedimiento se llegará a un punto que uno de los dos jugadores tendrá que sacrificar algún cuadrado, pues ya no hay más espacios disponibles, y entonces se completarán el resto rápidamente, al generarse un 'efecto en cadena'. Los buenos jugadores preverán cuándo comenzará este efecto y podrán adaptar sus movimientos a su favor.

2. Ejecución

El primer paso fue diseñar el juego en Python. Para ello, usamos el paquete *pygame*, puesto que se ajustaba exactamente a lo que buscábamos. Optamos por una paleta de colores simple, en la que el fondo sería gris, las aristas y los puntos negros, y los colores de los cuadrados de los correspondientes jugadores serán verde y rojo pastel. Primero creamos un programa en el que se enfrentarían dos usuarios, sin intervención de un bot. Cabe destacar que se ha decidido omitir una regla del juego original. Y es que si un jugador completa un cuadrado, normalmente recibe un turno extra. Sin embargo, con tal de preservar el modelo de turnos alternos, se ha decidido que se suprimirá. Una vez obtenido una interfaz satisfactoria, se probó repetidamente en busca de errores y se arreglaron.

Una vez hecho esto, ya teníamos la base. A partir de ahora se podía jugar *PvP* (Player vs Player). Sin emargo, nuestro objetivo es desarrollar una IA contra la que jugar. Necesitaríamos diseñar un algoritmo *Minimax* que explorase en profundidad el árbol de posibles jugadas, y que esta profundidad se pueda personalizar. Optamos por un algoritmo *Minimax* de poda alfa-beta, pues el árbol de posibles jugadas resulta gigante. Necesitamos podar ramas siempre y cuando sea posible, pues disponemos de capacidad de computación limitada. Por ello nos ajustamos al siguiente pseudocódigo.

Algorithm 1 Función Minimax(v:nodo, n:entero, α, β, turno)

Entrada: v: nodo, n: entero, α, β, turno
Salida : valor del minimax

```
var w: nodo; parar: boolean fin_var
si (n = 0 ∨ pos_moves(game)= ∅) entonces
  | Minimax ← eval(game)
en otro caso
  | parar ← Falso
  | si turno > 0 entonces
  |   | mientras (¬ parar) hacer
  |   |   | w ← sigsucesor(game)
  |   |   | α ← max(α, Minimax(w, n − 1, α, beta, turnomin))
  |   |   | si (β ≤ α ∨ w = ultimo) entonces
  |   |   |   | parar ← True
  |   |   | fin
  |   | fin
  |   | Minimax ← α
  | en otro caso
  |   | mientras (¬ parar) hacer
  |   |   | w ← sigsucesor(game)
  |   |   | β ← min(β, Minimax(w, n − 1, α, beta, turnomax))
  |   |   | si (β ≤ α ∨ w = ultimo) entonces
  |   |   |   | parar ← True
  |   |   | fin
  |   | fin
  |   | Minimax ← β
  | fin
fin
devolver Minimax
```

La función *eval* es un heurístico que nos ayudará a estimar el estado de una partida. Cuanto mayor sea el valor, más deseable será ese estado para el jugador en cuestión. Este era un tema de interés para nosotros, pues la elección de este heurístico no es para nada trivial. Por ello, quisimos probar con distintas posibilidades y posteriormente compararlas. Por un lado, hemos contabilizado el número de cuadrados del bot en un determinado momento y por el otro hemos tomado la diferencia de cuadrados de un jugador con respecto al otro, para así evitar también

que el oponente consiga puntos. A priori supusimos que el segundo sería mejor, pues tiene más variables en cuenta y posiblemente fuesen más significativas. Veremos más adelante si se confirma nuestra suposición.
También se ha de tener en cuenta que dependiendo de si el bot se considerase *Jugador 1* o *Jugador 2*, se tendría que cambiar el signo a esta función. Esto toma especial importancia a la hora de enfrentar los bots el uno contra el otro, pues nunca usarán la misma función, sino la negativa de esta.
Una vez construida y refinada la IA (con profundidad regulable), sólo nos quedaba ajustar el código para poder enfrentarla a sí misma. Se puede ajustar la profundidad de cada bot por separado para así poder examinar que heurístico es mejor, que era uno de nuestros objetivos.

3. Complejidad Temporal Teórica

En este apartado calcularemos la complejidad temporal tanto de las funciones auxiliares como de la función principal *minimax()* en función del parámetro n, que es la dimensión o cantidad de cuadrados del tablero. En lo siguiente la estimación será muy pesimista, tomando en mayor parte el peor caso, a no ser que se diga lo contrario.

3.1. Función *pos_moves()*

Esta función toma como valor de entrada el estado actual de la partida y devuelve una lista con todos los movimientos posibles, en forma de tupla. En rasgos generales, esta función recorre dos matrices de tamaño $n \times n+1$ y $n+1 \times n$, respectivamente, mientras comprueba si hay un 0 o un 1 en esa casilla. Entonces, la complejidad temporal tiene el siguiente aspecto.

$$\begin{aligned} T_{pos_moves}(n) &= 1 + \sum_{i=1}^{n+1}\sum_{j=1}^{n}(2) + \sum_{i=1}^{n}\sum_{j=1}^{n+1}(2) \\ &= 1 + 2(n+1)n + 2n(n+1) = 1 + 4n(n+1) = \Theta(n^2) \end{aligned}$$

3.2. Función *eval()*

Esta función evalúa el estado actual de la partida. Se han definido varias funciones heurísticas de este tipo, pero todas siguen el siguiente esquema general.

$$T_{eval}(n) = n + \sum_{i=1}^{n} n = n + n^2 = \Theta(n^2)$$

3.3. Función *Minimax()*

Esta función es la principal de nuestro bot. Usa las dos funciones anteriores y recursividad para calcular en un momento dado de la partida el mejor movimiento para cualquiera de los dos jugadores. Se supondrá un algoritmo sin poda para calcular una cota superior real, aunque no realista en la práctica. Asimismo, se omite el concepto de poda alfa-beta y se supone que el algoritmo recorre todo el árbol. También se supone que el costo de hacer una copia del juego es del orden de $O(n^2)$, pues tiene que copiar 3 matrices de tamaños $n \times n$, con pequeñas variaciones. El coste temporal de ella es el siguiente.

$$T_{minimax}(p,n) = \begin{cases} O(n^2) + n + C_1 + \sum_{i=1}^{q}(O(n^2) + T(p-1,n) + C_2) & \text{si } p \neq 0 \\ O(n^2) + n + O(n^2) & \text{si } p = 0 \end{cases}$$

Donde p es la profundidad a la que actuará el algoritmo y q indica el número de movimientos posibles, que se puede acotar superiormente por $2n(n+1)$, pues es el número de movimientos posibles al inicio de la partida, e irán disminuyendo en adelante. Simplificando y omitiendo las constantes, pues son insignificantes comparadas con n, obtenemos

$$T_{minimax}(p,n) = \begin{cases} 2n(n+1)O(n^2) + 2n(n+1)T(p-1,n) & \text{si } p \neq 0 \\ 2O(n^2) + n & \text{si } p = 0 \end{cases}$$

Entonces, expandiendo la recursión obtenemos

$$\begin{aligned} T(p,n) &= 2n(n+1)O(n^2) + 2n(n+1)T(p-1,n) = \\ &= 2n(n+1)O(n^2) + 2n(n+1)[2n(n+1)O(n^2) + 2n(n+1)T(p-2,n)] \\ &= 2n(n+1)O(n^2) + (2n(n+1))^2O(n^2) + (2n(n+1))^2T(p-2,n) \end{aligned}$$

$$= 2n(n+1)O(n^2) + (2n(n+1))^2 O(n^2) + (2n(n+1))^2[2n(n+1)O(n^2) + 2n(n+1)T(p-3,n)]$$

$$= 2n(n+1)O(n^2) + (2n(n+1))^2 O(n^2) + (2n(n+1))^3 O(n^2) + (2n(n+1))^3 T(p-3,n)$$

$$= \dots$$

$$= 2n(n+1)O(n^2) + (2n(n+1))^2 O(n^2) + \cdots + (2n(n+1))^k O(n^2) + (2n(n+1))^k T(p-k,n)$$

Introduciendo entonces $k = p$,

$$\begin{aligned} T(p,n) &= \sum_{i=1}^{p} (2n(n+1))^k O(n^2) + (2n(n+1))^p T(0,n) \\ &= \sum_{i=1}^{p} (2n(n+1))^k O(n^2) + (2n(n+1))^p [2O(n^2) + n] \\ &\leq (2n(n+1))^p O(n^2) + (2n(n+1))^p [2O(n^2) + n] \\ &= (2n(n+1))^p [3O(n^2) + n] \end{aligned}$$

Esta es una cota superior de la complejidad temporal, así que podemos decir con firmeza que el algoritmo *Minimax* tiene complejidad temporal de $O\left(n^{2p+2}\right)$.

4. Complejidad Espacial Teórica

Nos interesa también saber de cuánto almacenamiento deberemos disponer a la hora de ejecutar estos algoritmos. Para ello, estudiaremos la complejidad espacial.
El juego se almacena internamente en forma de tres matrices de tamaños $n \times n$, por lo cual almacenaremos $3n^2$ datos principales. La ejecución del juego no precisa de más estructuras de datos de este tipo, hasta que llegamos al algoritmo *Minimax*. Precisamente aquí creamos copias del juego para hacer cambios sin modificar la partida inicial, y esto obviamente aumenta considerablemente la memoria necesaria. Para ser más concretos, en cada ciclo del *While* se hace una copia del juego y se hacen p ciclos. Si bien es cierto que hay más variables que se guardan, como por ejemplo una matriz que indica qué cuadrados han sido completados o listas con los posibles movimientos, generadas por *pos_moves()*, su consumo nunca superará el orden de n^2, por lo cual hemos decidido omitirlas en lo sucesivo. En resumen, en el algoritmo se consumirán pn^2 unidades de memoria y se concluye pues que la complejidad espacial del algoritmo es del orden de $O(pn^2)$.

5. Análisis y Discusión

Hasta este punto hemos programado nuestra implementación en Python del juego *Dots and Boxes*. Hemos diseñado un algoritmo Minimax con poda alfa-beta, que admite diferentes funciones heurísticas de evaluación. A partir de ahora nos dedicaremos a analizar cómo rinde la IA en diferentes condiciones. Se va a analizar la influencia del tamaño del tablero, la profundidad examinada y la función de evaluación.

5.1. Tiempo

El primer punto de interés es el tiempo. Hemos calculado una complejidad temporal del orden de $O(n^{2p+2})$ y queremos ver si es plausible. Para ello, lo primero que vamos a hacer va a ser ejecutar el código múltiples veces y calcular la media del tiempo que tarda en llevar a cabo una partida de IA contra IA. Veremos como afecta en esto la dimensión del tablero (n), la profundidad (p) del la IA roja (Rp) y la de la IA verde (Gp).

n=2	G1	G2	G3	G4
R1	0,06243037	0,064937568	0,088509774	0,216751373
R2	0,053859115	0,071448994	0,092090595	0,224758828
R3	0,086445439	0,106057954	0,126488292	0,257757902
R4	0,164692044	0,180431366	0,193821669	0,336540234

Tiempos de partida con n=2

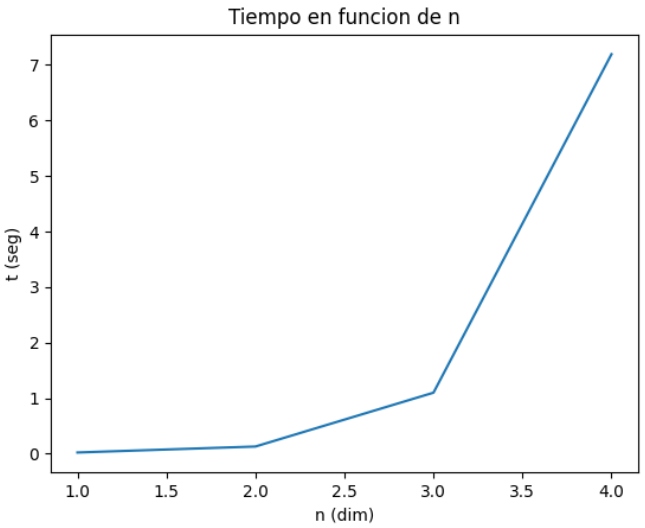
n=3	G1	G2	G3	G4
R1	0,111668646	0,329694974	0,589687347	4,344940305
R2	0,151271081	0,367803788	0,628122962	4,439124691
R3	0,624586391	0,855696118	1,095268631	4,90354017
R4	1,652452278	1,784414673	2,070077074	5,830212629

Tiempos de partida con n=3

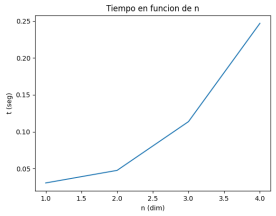
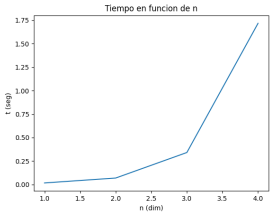
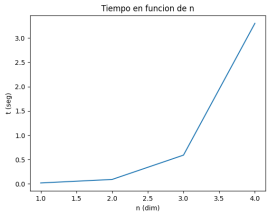
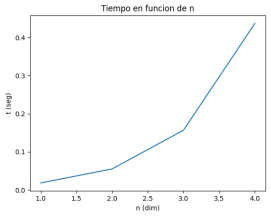
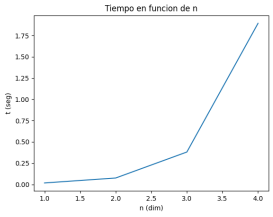
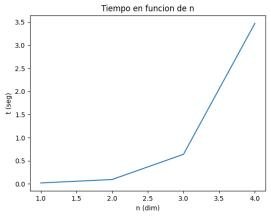
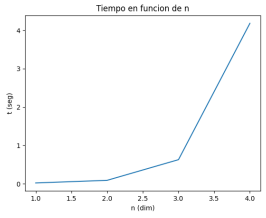
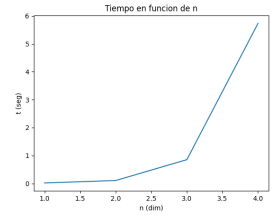
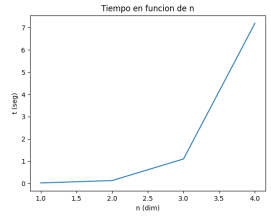
n=4	G1	G2	G3
R1	0,245628786	1,689854598	3,208082867
R2	0,42391299	1,838395023	3,490647602
R3	4,406897783	5,46723299	7,157749832

Tiempos de partida con n=4

Se ve que el tiempo sube muy rápido si ampliamos la profundidad o aumentamos la dimensión del tablero, lo cual encaja con nuestro coste temporal, que resulta muy alto. De todas formas conviene ver estos número en gráficos. Hagamos esto. Para ello hemos graficado los datos de la siguiente forma.



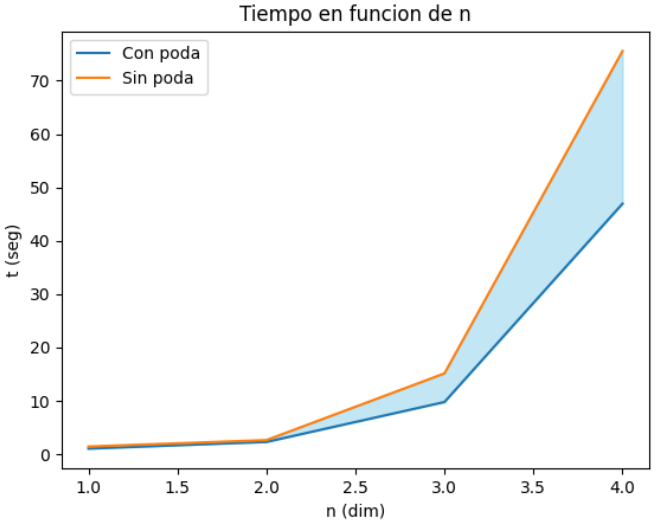
Como se ve, en el eje x hemos colocado la dimensión del tablero y en el eje y está el tiempo que se ha tardado en ejecutar, de media, una partida. Este era un ejemplo para profundidad 3 de ambos bots.

	G1	G2	G3
R1			
R2			
R3			

Tiempo en función de n

Viendo esto no resulta descabellado pensar que, en efecto, el crecimiento del tiempo es exponencial. Esto se ajusta a lo previamente calculado. Sería interesante poder extender los gráficos, pero no disponemos de la capacidad computacional para hacer esto, teniendo en cuenta que hay que ejecutar las partidas repetidas veces.

Además, también queremos ver si realmente es eficiente nuestra poda. Para ello, comprobamos el tiempo que se tarda en ejecutar 20 partidas con y sin poda. Obtenemos los siguientes resultados.



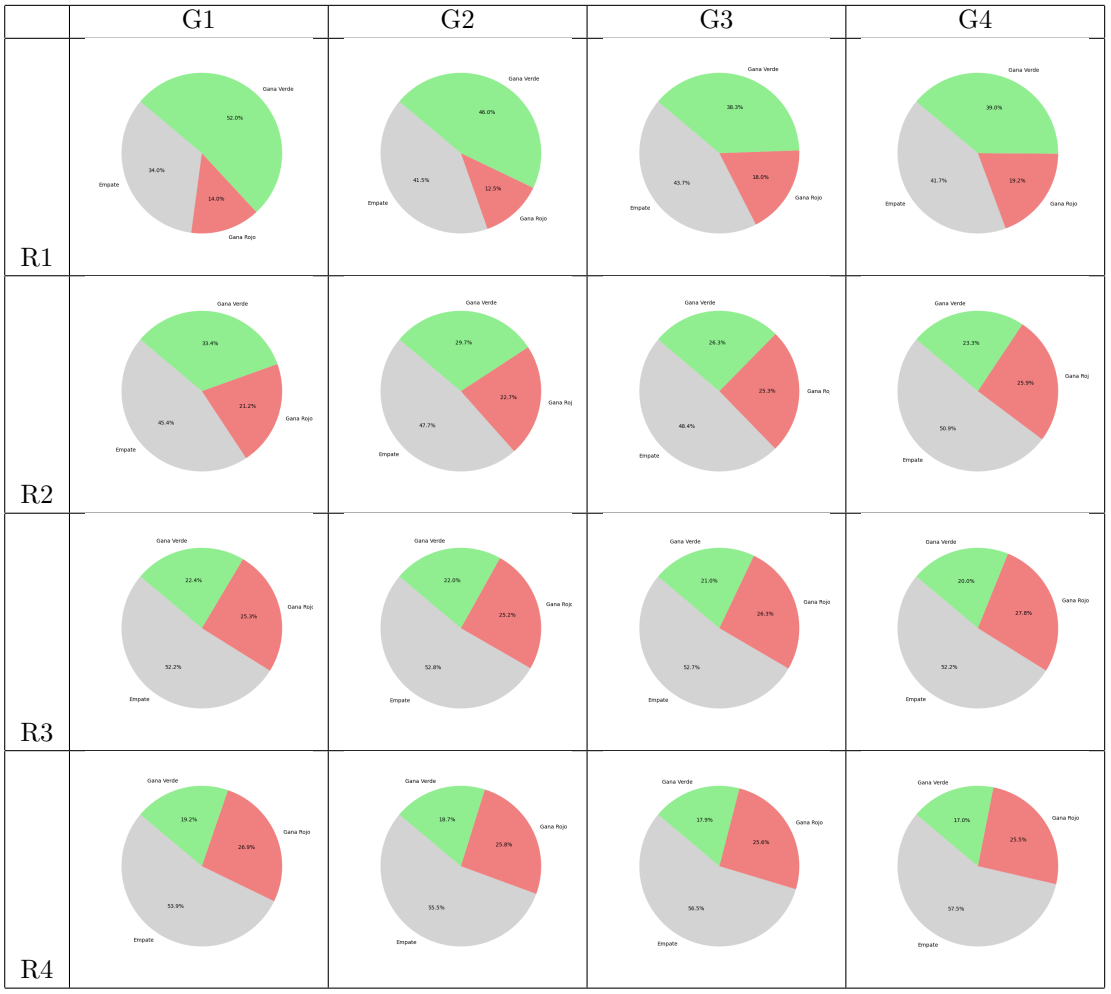
Comparación de tiempos con y sin poda para R2 y G2

Se ve claramente que la poda resulta muy efectiva, disminuyendo el tiempo a un 62,14 % en el caso de un tablero 4×4.

5.2. Profundidad

En este apartado examinaremos cómo afecta al rendimiento de la IA el número de capas que investigará en el algoritmo Minimax. Nuestra intuición nos lleva a pensar que a mayor profundidad mejor es el bot, pero veamos si realmente se cumple esto.
Para examinarlo, hemos enfrentado a las IAs 100 partidas seguidas y apuntado el ganador. Finalmente pusimos los resultados en un histograma circular. Repitiendo este proceso para las diferentes dimensiones del tablero y profundidades obtenemos las siguientes tablas.

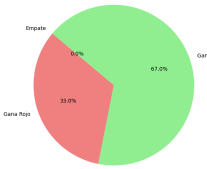
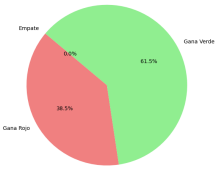
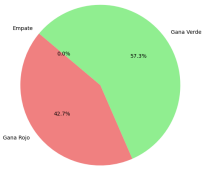
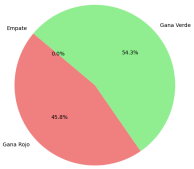
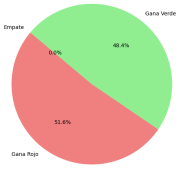
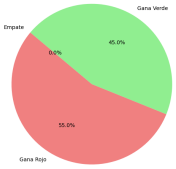
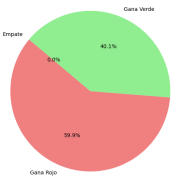
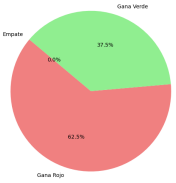
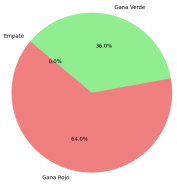
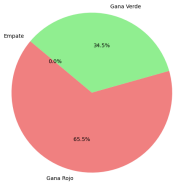
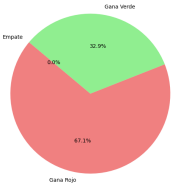
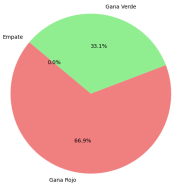
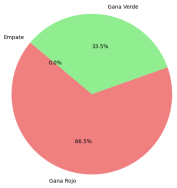
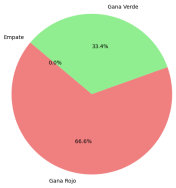
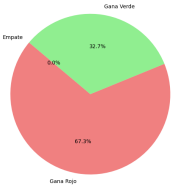
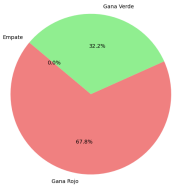
Dimensión 2



Tamaño del tablero: 2 cuadrados

Lo primero que llama la atención es que hay, por lo general muchos empates. Por lo general, aproximadamente la mitad de las partidas acaban en tablas. Si bien nos llama la atención, tampoco nos sorprende, porque al haber sólo dos cuadrados es muy fácil que cada jugador cierre dos de ellos, sobre todo sabiendo que a menudo cerrar uno da la oportunidad al rival de cerrar otro. Además, salta a a la vista que el bot verde no se beneficia tanto como el rojo de ampliar la profundidad. De hecho, si ambos tienen profundidad baja, por lo general gana el verde, pero si el rojo aumenta su profundidad gana más partidas que el verde, y no viceversa.
Por lo general, pensamos que en un tablero 2×2 no influye tanto la estrategia, pues los resultados son muy similares e influye mucho quién empiece.

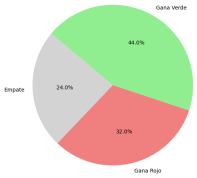
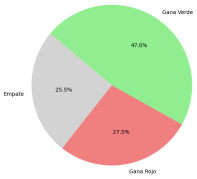
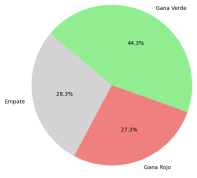
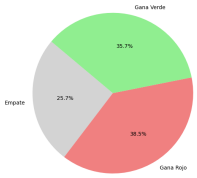
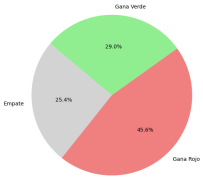
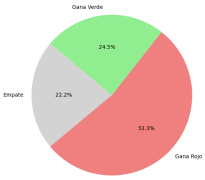
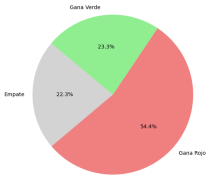
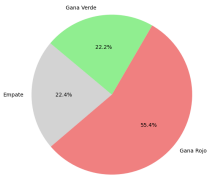
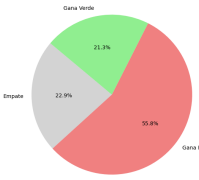
Dimensión 3

	G1	G2	G3	G4
R1				
R2				
R3				
R4				

Tamaño del tablero: 3 cuadrados

En el caso de un tablero 3×3 vemos cosas más curiosas. La primera es que no hay empates. Sin embargo esto tiene mucho sentido pues hay 9 cuadrados a completar en total, que es un número impar en contraste con los 4 de antes. Pero lo más impactante es que, por lo general, parece que el verde pierde tasa de victorias cuanta más profundidad examina. El rojo también presenta este comportamiento, a excepción de cuando pasa de profundidad 1 a 2, en cuyo caso aumenta drásticamente su tasa de victorias, lo cual era nuestra intuición a priori.

Dimensión 4

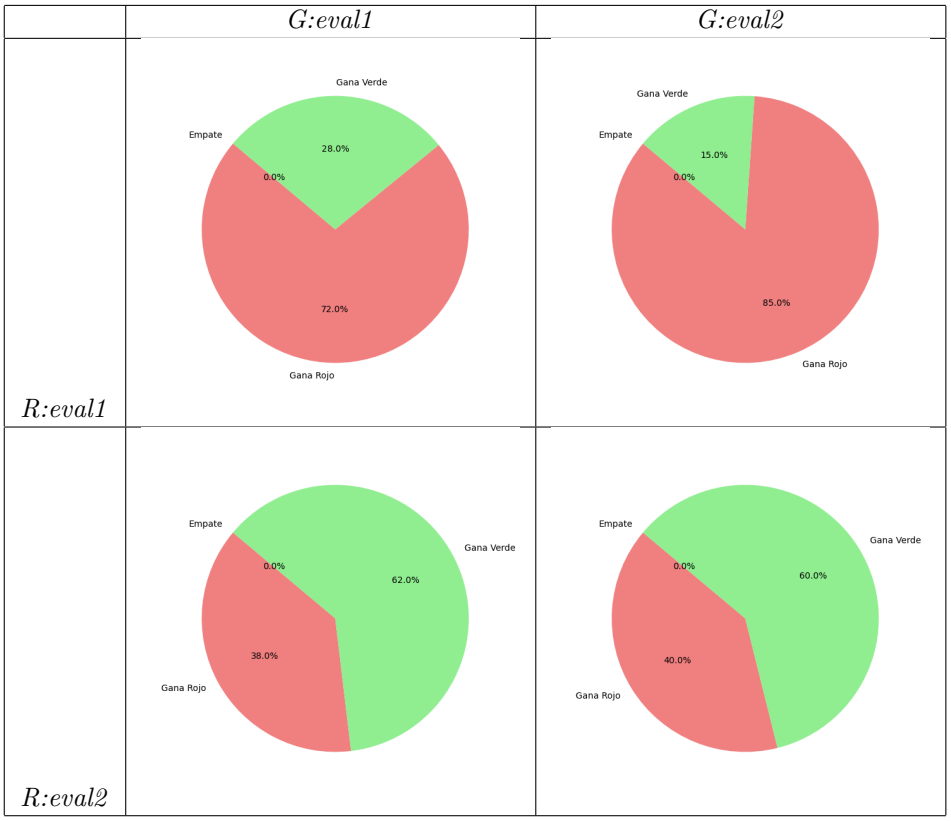
	G1	G2	G3
R1			
R2			
R3			

Tamaño del tablero: 4 cuadrados

Como tenemos un tablero 4×4, hemos tenido que recortar la profundidad de los bots y no llegaremos a 4, pues ya aumenta el árbol de posibilidades considerablemente y no disponemos de recursos computacionales suficientes para trabajar este problema. En esta tabla vemos empates de nuevo, como era de esperar, pues en un tablero 4×4 hay cuadrados pares, 16. En lineas generales, esta tabla se comporta mucho más como esperaríamos. Si ambos bots examinan sólo un movimiento, ganan en la misma proporción partidas. Además, parece que el bot rojo presenta una relación positiva entre la profundidad y la tasa de victorias. Esto es, a más profundidad gana más partidas. No obstante, el verde da la sensación de que, de nuevo, pierde ligeramente más partidas cuanta más profundidad examina.

5.3. Función *eval*

Por último queremos probar diferentes funciones de evaluación. Definiremos las funciones *eval1()* y *eval2()*. La primera es la función que hemos usado hasta ahora, la diferencia de cuadrados entre ambos jugadores. La segunda indicará solamente el número de cuadrados del jugador. Nuestra hipótesis es que funcionará mejor la función original, pues no importa si un jugador hace un número determinado de cuadrados si el oponente consigue más. En este caso



Evals en tablero 3×3

estamos en un tablero de dimensión 3. Hemos probado como rinden las funciones de evaluación por parejas. Vemos que si se usa la misma función de evaluación, el bot que inicia la partida se beneficia más de la función *eval1*, mientras que el que no tiene el primer turno prefiere que se use la segunda. En cambio, si se usan funciones de evaluación distintas, está más que claro que gana el que use la primera función en la mayoría de los casos.

6. Conclusiones

En esta práctica hemos podido trabajar en un ejemplo práctico de los algoritmos vistos durante el curso, en el que gracias a la máquina hemos podido determinar con éxito el que debería ser la mejor respuesta a cada posición.
La función de evaluación más óptima será la diferencia entre cuadrados, aunque para encontrar alguna que afine mucho más habría que tener en cuenta, quizás, cuadrados que estén a punto de ser completados a parte de los ya coloreados de un color u otro.
También hemos observado que no necesariamente tiene que ser mejor hacer un estudio en mucha profundidad en un algoritmo minimax, pues se necesita mucho tiempo y no necesariamente aumenta la tasa de victorias.
La utilización de la poda, sobre todo para niveles de profundidad mayores es sin duda necesaria y muy útil, pues reducimos la cantidad de operaciones significativamente.

7. Anexo: Código Python

```python
import pygame, sys, random, copy
import tkinter as tk


###################### CLASE JUEGO ######################

class DotsAndBoxes:

    def __init__(self, n, screen):
        self.n = n
        self.horizontal_lines = [[False] * n for _ in range(n + 1)]
        self.vertical_lines = [[False] * (n + 1) for _ in range(n)]
        self.square_completed = [[0] * self.n for _ in range(self.n)]
        self.screen = screen
        self.width = self.screen.get_width() * 0.8
        self.height = self.screen.get_height() * 0.8
        self.gap = self.width // (n + 1)
        self.total_squares = 0
        self.turn = 0

    def copy(self):
        gamecopy = DotsAndBoxes(self.n, self.screen)
        gamecopy.horizontal_lines = copy.deepcopy(self.horizontal_lines)
        gamecopy.vertical_lines = copy.deepcopy(self.vertical_lines)
        gamecopy.square_completed = copy.deepcopy(self.square_completed)
        gamecopy.total_squares = self.total_squares
        gamecopy.turn = self.turn
        return gamecopy

    def draw_horizontal_line(self, row, col):
        if 0 <= row <= self.n and 0 <= col <= self.n - 1:
            if not self.horizontal_lines[row][col]:
                self.horizontal_lines[row][col] = True
                # Check adjacent squares to the horizontal line
                if 0 < row <= self.n:
                    if (self.horizontal_lines[row - 1][col], self.vertical_lines[row
                        self.vertical_lines[row - 1][col + 1]) == (True, True, True
                        self.square_completed[row - 1][col] = self.turn + 1
                        self.total_squares += 1
                if 0 <= row < self.n:
                    if (self.horizontal_lines[row + 1][col], self.vertical_lines[row
                        self.vertical_lines[row][col + 1]) == (True, True, True):
                        self.square_completed[row][col] = self.turn + 1
                        self.total_squares += 1
                self.turn = (self.turn + 1) % 2
                return True
        self.turn = (self.turn + 1) % 2
        return False

    def draw_vertical_line(self, row, col):
        if 0 <= row <= self.n - 1 and 0 <= col <= self.n:
            if not self.vertical_lines[row][col]:
                self.vertical_lines[row][col] = True
```

```
                # Check adjacent squares to the vertical line
                if 0 < col <= self.n:
                    if (self.vertical_lines[row][col - 1], self.horizontal_lines[row
                        self.horizontal_lines[row + 1][col - 1]) == (True, True, Tr
                        self.square_completed[row][col - 1] = self.turn + 1
                        self.total_squares += 1
                if 0 <= col < self.n:
                    if (self.vertical_lines[row][col + 1], self.horizontal_lines[row
                        self.horizontal_lines[row + 1][col]) == (True, True, True):
                        self.square_completed[row][col] = self.turn + 1
                        self.total_squares += 1
                self.turn = (self.turn + 1) % 2
                return True
        self.turn = (self.turn + 1) % 2
        return False

    def draw_board(self):
        # Clear the screen
        self.screen.fill((229, 229, 229))

        for row in range(self.n):  # Squares
            for col in range(self.n):
                if self.square_completed[row][col] == 1:
                    color = (236, 112, 99)  # Red
                    pygame.draw.rect(self.screen, color,
                                     (self.gap * (col + 1), self.gap * (row + 1), s
                elif self.square_completed[row][col] == 2:
                    color = (88, 214, 141)  # Green
                    pygame.draw.rect(self.screen, color,
                                     (self.gap * (col + 1), self.gap * (row + 1), s

        for row in range(self.n + 1):
            for col in range(self.n):
                if self.horizontal_lines[row][col]:
                    pygame.draw.line(self.screen, (0, 0, 0), (self.gap * (col + 1),
                                     (self.gap * (col + 2), self.gap * (row + 1)),
        for row in range(self.n):
            for col in range(self.n + 1):
                if self.vertical_lines[row][col]:
                    pygame.draw.line(self.screen, (0, 0, 0), (self.gap * (col + 1),
                                     (self.gap * (col + 1), self.gap * (row + 2)),

        for row in range(self.n + 1):  # Circles
            for col in range(self.n + 1):
                pygame.draw.circle(self.screen, (0, 0, 0), (self.gap * (col + 1), s

        red_squares = sum(row.count(1) for row in self.square_completed)
        blue_squares = sum(row.count(2) for row in self.square_completed)
        font = pygame.font.SysFont(None, 24)
        players = dict()
        players[0] = "Red"
        players[1] = "Green"

        text_surface = font.render(
            f"Red: {red_squares}, Green: {blue_squares}, Turn: {players[self.turn]
```

```
            True,
            (0, 0, 0))
        self.screen.blit(text_surface, (10, 10))

    def process_click(self, pos):  # Falla si le damos a la derecha / abajo de la l
        x, y = pos
        row = round(y / self.gap) - 1
        col = round(x / self.gap) - 1
        if min(abs(y % self.gap), self.gap - abs(y % self.gap)) < 5:
            self.draw_horizontal_line(row, col)
            return True
        elif min(abs(x % self.gap), self.gap - abs(x % self.gap)) < 5:
            self.draw_vertical_line(row, col)
            return True
        return False

    def process_move(self, tuple):
        if tuple[0] == 0:
            self.draw_horizontal_line(tuple[1], tuple[2])
        elif tuple[0] == 1:
            self.draw_vertical_line(tuple[1], tuple[2])

    def eval_green(self, lista): # Funci n de eval que se usa para el verde
        red_squares = sum(row.count(1) for row in lista)
        blue_squares = sum(row.count(2) for row in lista)
        return (red_squares - blue_squares) * (-1)

    def eval_red(self, lista): # Funci n de eval que se usa para el rojo
        red_squares = sum(row.count(1) for row in lista)
        blue_squares = sum(row.count(2) for row in lista)
        return red_squares - blue_squares

    def eval2(self, turn):  # Difference in completed squares, but own squares are
        red_squares = sum(row.count(1) for row in self.square_completed)
        blue_squares = sum(row.count(2) for row in self.square_completed)
        if turn == 0:
            red_squares = red_squares * 2
        else:
            blue_squares = blue_squares * 2
        return (red_squares - blue_squares) * ((-1) ** (turn))

    def eval1(self, turn):  # Cuadrados del contrario
        squares = sum(row.count(turn + 1) for row in self.square_completed)
        return squares

    def pos_moves(self):  # Return list of possible moves
        moves = []
        for i in range(self.n + 1):
            for j in range(self.n):
                if not self.horizontal_lines[i][j]:
                    moves.append((0, i, j))
        for i in range(self.n):
            for j in range(self.n + 1):
                if not self.vertical_lines[i][j]:
                    moves.append((1, i, j))
```

```
        return moves

    def Minimax(self, n, alfa, beta, turn, func_eval, back=None):
        if back is None:
            back = []
        moves = self.pos_moves()
        random.shuffle(moves)

        # print("Depth:", n, "Turn:", turn)
        if n == 0 or not moves:
            return func_eval(self.square_completed), back[0] if back else None

        best_move = None

        if turn == 0:  # Player's turn
            for move in moves:
                new = self.copy()
                new.process_move(move)

                lista = copy.deepcopy(back)
                lista.append(move)
                # print("Move:", move, "List:", lista)

                val, inicial = new.Minimax(n - 1, alfa, beta, new.turn, func_eval,
                # print(val)
                if val > alfa:
                    alfa = val
                    best_move = inicial

                if beta <= alfa:
                    break

            # print("Best Move (Max):", best_move, "alfa:", alfa)
            return alfa, best_move

        else:  # Opponent's turn
            for move in moves:
                new = self.copy()
                new.process_move(move)

                lista = copy.deepcopy(back)
                lista.append(move)

                # print("Move:", move, "List:", lista)

                val, inicial = new.Minimax(n - 1, alfa, beta, new.turn, func_eval,
                # print(val)

                if val < beta:
                    beta = val
                    best_move = inicial

                if beta <= alfa:
                    break

```

```
            # print("Best Move (Min):", best_move, "beta:", beta)
            return beta, best_move


class GameSetupWindow_PvsIA:
    def __init__(self):
        self.window = tk.Tk()
        self.window.title("Configuraci n del juego")

        # Etiqueta y botones para el tama o del tablero
        self.board_size_label = tk.Label(self.window, text="Tama o del tablero:")
        self.board_size_label.grid(row=0, column=0, padx=5, pady=5)

        self.board_size_buttons_frame = tk.Frame(self.window)
        self.board_size_buttons_frame.grid(row=0, column=1)
        self.board_size_buttons = []
        for size in [2, 3, 4]:
            button = tk.Button(self.board_size_buttons_frame, text=str(size),
                               command=lambda s=size: self.set_board_size(s))
            button.pack(side=tk.LEFT, padx=5, pady=5)
            self.board_size_buttons.append(button)

        # Etiqueta y botones para la profundidad del tablero
        self.depth_label = tk.Label(self.window, text="Profundidad de la IA:")
        self.depth_label.grid(row=1, column=0, padx=5, pady=5)

        self.depth_buttons_frame = tk.Frame(self.window)
        self.depth_buttons_frame.grid(row=1, column=1)
        self.depth_buttons = []
        for size in [1, 2, 3, 4, 5]:
            button = tk.Button(self.depth_buttons_frame, text=str(size),
                               command=lambda s=size: self.set_depth(s))
            button.pack(side=tk.LEFT, padx=5, pady=5)
            self.depth_buttons.append(button)

        # Bot n para comenzar el juego
        self.start_button = tk.Button(self.window, text="Comenzar juego", command=s
        self.start_button.grid(row=2, columnspan=2, padx=5, pady=5)

        self.board_size = None
        self.depth = None

        self.center_window()

    def set_board_size(self, size):
        self.board_size = size

    def set_depth(self, size):
        self.depth = size

    def start_game(self):
        self.window.destroy()  # Cerrar la ventana de configuraci n

    def center_window(self):
        self.window.update_idletasks()
```

```
        width = self.window.winfo_width()
        height = self.window.winfo_height()
        x = (self.window.winfo_screenwidth() // 2) - (width // 2)
        y = (self.window.winfo_screenheight() // 2) - (height // 2)
        self.window.geometry('{}x{}+{}+{}'.format(width, height, x, y))


class GameSetupWindow_IAvsIA:
    def __init__(self):
        self.window = tk.Tk()
        self.window.title("Configuraci n del juego")

        # Etiqueta y botones para el tama o del tablero
        self.board_size_label = tk.Label(self.window, text="Tama o del tablero:")
        self.board_size_label.grid(row=0, column=0, padx=5, pady=5)

        self.board_size_buttons_frame = tk.Frame(self.window)
        self.board_size_buttons_frame.grid(row=0, column=1)
        self.board_size_buttons = []
        for size in [2, 3, 4]:
            button = tk.Button(self.board_size_buttons_frame, text=str(size),
                               command=lambda s=size: self.set_board_size(s))
            button.pack(side=tk.LEFT, padx=5, pady=5)
            self.board_size_buttons.append(button)

        # Etiqueta y botones para la profundidad del tablero
        self.depth_label = tk.Label(self.window, text="Profundidad IA Roja:")
        self.depth_label.grid(row=1, column=0, padx=5, pady=5)

        self.depth_buttons_frame = tk.Frame(self.window)
        self.depth_buttons_frame.grid(row=1, column=1)
        self.depth_buttons = []
        for size in [1, 2, 3, 4, 5]:
            button = tk.Button(self.depth_buttons_frame, text=str(size),
                               command=lambda s=size: self.set_depth(s))
            button.pack(side=tk.LEFT, padx=5, pady=5)
            self.depth_buttons.append(button)

        # Etiqueta y botones para la profundidad 2
        self.depth2_label = tk.Label(self.window, text="Profundidad IA Verde:")
        self.depth2_label.grid(row=2, column=0, padx=5, pady=5)

        self.depth2_buttons_frame = tk.Frame(self.window)
        self.depth2_buttons_frame.grid(row=2, column=1)
        self.depth2_buttons = []
        for size in [1, 2, 3, 4, 5]:
            button = tk.Button(self.depth2_buttons_frame, text=str(size),
                               command=lambda s=size: self.set_depth2(s))
            button.pack(side=tk.LEFT, padx=5, pady=5)
            self.depth2_buttons.append(button)

        # Bot n para comenzar el juego
        self.start_button = tk.Button(self.window, text="Comenzar juego", command=s
        self.start_button.grid(row=3, columnspan=2, padx=5, pady=5)

```

```
        self.board_size = None
        self.depth = None
        self.depth2 = None

        self.center_window()

    def set_board_size(self, size):
        self.board_size = size

    def set_depth(self, depth):
        self.depth = depth

    def set_depth2(self, depth):
        self.depth2 = depth

    def start_game(self):
        self.window.destroy()

    def center_window(self):
        self.window.update_idletasks()
        width = self.window.winfo_width()
        height = self.window.winfo_height()
        x = (self.window.winfo_screenwidth() // 2) - (width // 2)
        y = (self.window.winfo_screenheight() // 2) - (height // 2)
        self.window.geometry('{}x{}+{}+{}'.format(width, height, x, y))


# Funci n para crear la ventana de selecci n de modo de juego

def ventana_modo_juego():
    ventana = tk.Tk()
    ventana.title("Seleccionar Modo de Juego")

    # Obtener las dimensiones de la pantalla
    ancho_pantalla = ventana.winfo_screenwidth()
    altura_pantalla = ventana.winfo_screenheight()

    # Dimensiones de la ventana
    ancho_ventana = 300
    altura_ventana = 200

    # Calcular la posici n para centrar la ventana
    x = (ancho_pantalla // 2) - (ancho_ventana // 2)
    y = (altura_pantalla // 2) - (altura_ventana // 2)

    # Establecer las dimensiones y posici n de la ventana
    ventana.geometry(f"{ancho_ventana}x{altura_ventana}+{x}+{y}")

    modo_seleccionado = tk.IntVar()  # Variable para almacenar el modo seleccionado

    # Funci n para manejar la selecci n de modo de juego
    def on_click(modo):
        modo_seleccionado.set(modo)
        ventana.destroy()

```

```
    # Crear cajas clicables para los modos de juego
    modos = [("IA vs IA", 0), ("Jugador vs IA", 1), ("Jugador 1 vs Jugador 2", 2)]
    for modo, valor in modos:
        boton = tk.Button(ventana, text=modo, command=lambda valor=valor: on_click(
        boton.pack(pady=5)

    ventana.mainloop()

    return modo_seleccionado.get() # # Elegir modo de juego



###################### DIFERENTES MODOS DE JUEGO ######################


def main():  # Main function for AI vs AI
    setup_window = GameSetupWindow_IAvsIA()
    setup_window.window.mainloop()
    pygame.init()

    size = (1000, 1000)  # Window size
    screen = pygame.display.set_mode(size)
    pygame.display.set_caption("Dots and Boxes")  # Window title

    n = setup_window.board_size  # Board size (squares)
    profundidad1 = setup_window.depth # profundidad del player 1 RED
    profundidad2 = setup_window.depth2 # profundidad del player 2 GREEN

    game = DotsAndBoxes(n, screen)

    running = True
    while running:
        for event in pygame.event.get():  # Event handling loop
            if event.type == pygame.QUIT:
                running = False  # Exit the main loop if the user quits the game
                pygame.quit()
                sys.exit()
        game.draw_board()  # Draw the board
        pygame.display.flip()  # Update the display

        while running:

            # Player 1 (red)
            new = game.copy()  # Copiamos el juego para que no fastidie nuestras va
            new.turn = 0
            minimax_val, best_move = game.Minimax(profundidad1, -99999, 99999, new.
                                                  new.eval_red)  # Calculate AI's m
            game.process_move(best_move)  # Process AI's move
            game.draw_board()  # Draw the updated board
            pygame.display.flip()  # Update the display

            if game.total_squares == n * n:  # if game is finished then close the w
                pygame.time.wait(2000)
                running = False

```

```
            # Player 2 (green)
            pygame.time.wait(1)  # Wait a bit before the next move

            new = game.copy()  # Copiamos el juego para que no fastidie nuestras va
            new.turn = 0
            minimax_val, best_move = game.Minimax(profundidad2, -99999, 99999, new.
                                                  new.eval_green)  # Calculate AI's
            game.process_move(best_move)  # Process AI's move
            game.draw_board()  # Draw the updated board
            pygame.display.flip()  # Update the display

            pygame.time.wait(1)

            if game.total_squares == n * n:  # if game is finished then close the w
                m = game.square_completed
                contr=0
                contg=0
                for fila in m:
                    for cas in fila:
                        if cas == 1:
                            contr += 1
                        elif cas == 2:
                            contg += 1
                if contg < contr:
                    winner = 1
                elif contg > contr:
                    winner = 2
                elif contg == contr:
                    winner = 0
                pygame.time.wait(2000)
                return winner
                running = False

    #pygame.quit()  # Quit Pygame


def main2():  # Player vs Player
    pygame.init()
    size = (1000, 1000)  # Window size
    screen = pygame.display.set_mode(size)
    pygame.display.set_caption("Dots and Boxes")  # Window title
    n = 2  # Board size (squares)
    game = DotsAndBoxes(n, screen)
    running = True

    while running:
        valid_click = False
        game.draw_board()
        pygame.display.flip()
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
                pygame.quit()
                sys.exit()
```

```
            elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
# Left click
                valid_click = game.process_click(pygame.mouse.get_pos())
# Process mouse click for player 1

        if valid_click:  # If a valid click was made, then draw the board and updat
            game.draw_board()
            pygame.display.flip()
            if game.total_squares == n * n:
                m = game.square_completed
                contr=0
                contg=0
                for fila in m:
                    for cas in fila:
                        if cas == 1:
                            contr += 1
                        elif cas == 2:
                            contg += 1
                if contg < contr:
                    winner = 1
                elif contg > contr:
                    winner = 2
                elif contg == contr:
                    winner = 0
                pygame.time.wait(2000)
                return winner
                running = False


def main3():  # Player vs AI
    setup_window = GameSetupWindow_PvsIA()
    setup_window.window.mainloop()
    pygame.init()

    size = (1000, 1000)  # Window size
    screen = pygame.display.set_mode(size)
    pygame.display.set_caption("Dots and Boxes")  # Window title

    n = setup_window.board_size  # Board size (squares)
    profundidad = setup_window.depth

    # Si no le doy click bien a la ventanita pues pongo 2 por la cara

    if n is None:
        n = 2
    if profundidad is None:
        profundidad = 2

    game = DotsAndBoxes(n, screen)
    running = True

    while running:
        valid_click = False
        game.draw_board()
        pygame.display.flip()
```

```
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
                pygame.quit()
                sys.exit()
            elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
# Click izquierdo
                valid_click = game.process_click(pygame.mouse.get_pos())
# Procesa el click

        if valid_click:  # Aqu  hay un bug, porque si hago click se cambia de turno
            game.draw_board()
            pygame.display.flip()
            new = game.copy()  # Copiamos el juego para que no fastidie nuestras va
            new.turn = 0  # Establezco el turno 0 porque queremos que juegue al MAX
            minimax_val, best_move = new.Minimax(profundidad, -99999, 99999, new.tu
                                                 game.eval_red)  # Calculamos el mo
            if game.total_squares != n * n:  # Comprueba a ver si se ha terminado
                game.process_move(best_move)
                game.draw_board()
                pygame.display.flip()
            else:
                m = game.square_completed
                contr=0
                contg=0
                for fila in m:
                    for cas in fila:
                        if cas == 1:
                            contr += 1
                        elif cas == 2:
                            contg += 1
                if contg < contr:
                    winner = 1
                elif contg > contr:
                    winner = 2
                elif contg == contr:
                    winner = 0
                pygame.time.wait(1)
                return winner
                running = False



def mostrar_ganador(color):
    pantalla = pygame.display.set_mode((800, 400))
    pygame.display.set_caption("Ganador")

    if color == 1:
        color_texto = (236,112,99)  # Rojo
        mensaje = "Ganador es Rojo"
    elif color == 2:
        color_texto = (88,214,141)  # Verde
        mensaje = "Ganador es Verde"
    elif color == 0:
        color_texto = (0,0,0)
```

```
            mensaje = "Empate"
        else:
            pygame.quit()
            raise ValueError("Error en ganador")

        fuente = pygame.font.SysFont(None, 80)
        texto = fuente.render(mensaje, True, color_texto)

        pantalla.fill((229,229,229))
        pantalla.blit(texto, (170,160))
        pygame.display.flip()

        while True:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    pygame.quit()
                    sys.exit()

# Ejemplo de uso:
# mostrar_ganador(0)  # Muestra "Ganador es Rojo" en rojo
# mostrar_ganador(1)  # Muestra "Ganador es Verde" en verde


##########################################################################

if __name__ == "__main__":
    gamemode = ventana_modo_juego()
    # gamemode = int(input("0 si AI vs AI, 1 si Player vs AI, 2 si Player vs Player
    if gamemode == 0:
        winner = main()
        mostrar_ganador(winner)
    elif gamemode == 1:
        winner = main3()
        mostrar_ganador(winner)
    elif gamemode == 2:
        winner = main2()
        mostrar_ganador(winner)
```